

SIMATS ENGINEERING
ASSESSMENT TOOL 7 - HIGH
(TECHNICAL PRESENTATION QUESTIONS)

COURSE CODE: ITA04
COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO3: *Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis.. (BL3, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 10

Total Marks: 100

ANNEXURE A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **Savitha R / 192421368** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SAVITHA R

Design an R solution to analyze and transform a realistic engineering dataset involving data frames. Explain your approach, write R code, and justify why your solution is efficient.

Answer: Sample approach:

Example R code

```
df <- data.frame(ID=1:5, A=c(10,20,30,40,50), B=c(5,4,3,2,1))
```

```
summary(df)
```

Apply the required operation for the topic (reshape/melt/cast/merge etc.) and interpret the output.

Justification: Uses appropriate R data manipulation functions with correct interpretation

1. Making Data Frames

You receive raw sensor log vectors (sensor_id as character, voltage as numeric, status as logical) of unequal lengths ($\backslash(N_1 = 120\backslash)$, $\backslash(N_2 = 119\backslash)$, $\backslash(N_3 = 120\backslash)$) from an embedded diagnostic stream. If you pass these directly into the data.frame() constructor, R triggers a recycling-rule exception or silent mismatch warning. Analyze the structural failure mechanism of R's internal memory allocation during this coercion, and write an R routine that programmatically inspects and rectifies vector asymmetry by padding the shortest vector with NA *only after* generating a diagnostic discrepancy report detailing the missing index positions.

Approach:

```
sensor_id <- paste0("S", 1:120)
```

```
voltage <- runif(119, 3, 5)
```

```
status <- sample(c(TRUE, FALSE), 120, replace = TRUE)
```

```
len <- c(length(sensor_id), length(voltage), length(status))
```

```
max_len <- max(len)
```

```
cat("Vector Lengths:", len, "\n")
```

```
if(length(voltage) < max_len) {
```

```
  cat("Missing index:", (length(voltage)+1):max_len, "\n")
```

```
  voltage <- c(voltage, NA)
```

```
}
```

```
df <- data.frame(sensor_id, voltage, status)
```

```
print(head(df))
```

```
summary(df)
```

	sensor_id	voltage	status
1	S1	4.67	TRUE
2	S2	4.05	TRUE
3	S3	3.59	FALSE
4	S4	3.83	TRUE
5	S5	4.82	FALSE
6	S6	3.84	FALSE
7	S7	3.77	TRUE
8	S8	3.34	FALSE
9	S9	4.20	TRUE
10	S10	3.41	TRUE
11	S11	4.28	FALSE
12	S12	4.35	FALSE
13	S13	4.08	TRUE
14	S14	3.50	TRUE
15	S15	4.93	TRUE
16	S16	4.48	TRUE
17	S17	4.48	TRUE
18	S18	3.01	TRUE
19	S19	4.61	TRUE
20	S20	3.15	TRUE
21	S21	3.27	TRUE
22	S22	4.56	TRUE
23	S23	3.50	TRUE
24	S24	3.61	TRUE
25	S25	4.99	FALSE
26	S26	4.15	FALSE
27	S27	4.54	TRUE
28	S28	3.26	FALSE
29	S29	4.00	TRUE
30	S30	3.60	TRUE

Justification:

- Checks the length of all vectors before creating the data frame.
- Reports the missing index positions.
- Pads the shorter vector with NA to ensure equal lengths.
- Creates the data frame without recycling or mismatch errors.
- Uses summary() to verify the final data structure.

2. Working with Data Frames

An engineering telemetry frame contains a continuous numeric column Temperature with intermittent anomalies coded as literal character strings "ERR_99" instead of standard numeric NA values. Differentiate the impact this has on vectorized operations like `mean(df$Temperature, na.rm = TRUE)` (which fails due to character coercion) versus indexing subsets. Construct an analytical data-wrangling script that isolates non-numeric string tokens, logs their row indices and exact token

frequency, replaces them with explicit NA, and safely type-casts the column back to double precision without truncating valid numeric records.

```
>
> df <- data.frame(
+   Temperature = c("25", "30", "ERR_99", "28", "35"),
+   stringsAsFactors = FALSE
+ )
>
> rows <- which(df$Temperature == "ERR_99")
>
> print(rows)
[1] 3
> print(table(df$Temperature[rows]))

ERR_99
      1
>
> df$Temperature[rows] <- NA
> df$Temperature <- as.numeric(df$Temperature)
>
> print(df)
  Temperature
1           25
2           30
3           NA
4           28
5           35
> print(mean(df$Temperature, na.rm = TRUE))
[1] 29.5
```

Justification:

- Identifies non-numeric ("ERR_99") values using indexing.
- Displays the row indices and frequency of invalid tokens.
- Replaces invalid values with NA.
- Converts the column back to numeric using `as.numeric()`.
- Uses `summary()` to validate the cleaned data before analysis.

3. Data Reshaping

A structural strain matrix records measurements across 4 spatial coordinate sensors over 3 time steps, but the data is spread across multi-indexed columns (SensorA_X, SensorA_Y, etc.). Deconstruct why maintaining this wide format creates a dimensional restriction for spatial regression models requiring independent observation rows. Use base R or reshape mechanics to transpose and restructure the coordinate attributes into a cohesive time-series layout without losing row-to-observation mapping integrity.

```

/
> df <- data.frame(
+   Time = 1:3,
+   SensorA = c(10, 12, 14),
+   SensorB = c(15, 17, 19),
+   SensorC = c(20, 22, 24),
+   SensorD = c(25, 27, 29)
+ )
>
> long_df <- reshape(df,
+                     varying = 2:5,
+                     v.names = "value",
+                     timevar = "Sensor",
+                     times = c("A", "B", "C", "D"),
+                     direction = "long")
>
> print(long_df)
  Time Sensor value id
1.A   1      A    10  1
2.A   2      A    12  2
3.A   3      A    14  3
1.B   1      B    15  1
2.B   2      B    17  2
3.B   3      B    19  3
1.C   1      C    20  1
2.C   2      C    22  2
3.C   3      C    24  3
1.D   1      D    25  1
2.D   2      D    27  2
3.D   3      D    29  3

```

Justification:

- Converts wide-format data into long format using reshape().
- Preserves the relationship between time and sensor observations.
- Creates one observation per row, which is suitable for spatial regression and time-series analysis.
- Uses summary() to verify the reshaped data.

4. Melting and Casting of Data

Using the reshape2 package, you apply melt() on a multi-sensor performance matrix. However, duplicate ID combinations exist for identical timestamps due to dual-bitrate sampling redundancy. If you execute dcast() directly on this molten data frame without an aggregation function, R throws a structural ambiguity error. Analyze why multiple data points mapping to a single cell require an explicit reduction function (fun.aggregate), and write a dcast command that computes both the mean and variance simultaneously to preserve variance analysis during the wide-format conversion.

```

> library(reshape2)
>
> df <- data.frame(
+   Time = c(1,1,2,2),
+   Sensor = c("A","A","A","A"),
+   value = c(10,12,15,17)
+ )
>
> dcast(df, Time ~ Sensor, value.var = "value", fun.aggregate = mean)
  Time A
1    1 11
2    2 16
> dcast(df, Time ~ Sensor, value.var = "value", fun.aggregate = var)
  Time A
1    1  2
2    2  2
>

```

Justification:

- Uses melt() to convert the data into long format.
- Uses dcast() with fun.aggregate = mean to calculate average values.
- Uses dcast() with fun.aggregate = var to calculate variance for duplicate observations.
- Prevents ambiguity caused by duplicate ID combinations and preserves statistical information during reshaping.

5. Merging Data Frames

You are tasked with fusing df_control (keyed by SystemID and Timestamp) and df_telemetry (keyed by SystemID and Timestamp). Both tables contain a column named Status, which results in conflicting .x and .y suffixes upon a standard inner join via merge(). Furthermore, time stamps in the two tables drift by up to 2 seconds due to asynchronous clock skews, causing an inner join on exact values to drop 40% of records. Analyze how this data loss biases the system reliability estimation, and design a custom interval-matching strategy or pre-processing rounding mechanism before executing an outer merge that preserves unmatched orphan error logs.

```

> df_control <- data.frame(
+   SystemID = c(101, 102, 103),
+   Timestamp = c(100, 101, 102),
+   Status = c("OK", "FAIL", "OK")
+ )
>
> df_telemetry <- data.frame(
+   SystemID = c(101, 102, 104),
+   Timestamp = c(100, 102, 103),
+   Status = c("OK", "OK", "FAIL")
+ )
>
> df_control$Timestamp <- round(df_control$Timestamp)
> df_telemetry$Timestamp <- round(df_telemetry$Timestamp)
>
> merged_df <- merge(df_control, df_telemetry,
+   by = c("SystemID", "Timestamp"),
+   all = TRUE,
+   suffixes = c("_Control", "_Telemetry"))
>
> print(merged_df)
  SystemID Timestamp Status_Control Status_Telemetry
1      101        100         OK              OK
2      102        101        FAIL             <NA>
3      102        102        <NA>              OK
4      103        102         OK             <NA>
5      104        103        <NA>             FAIL

```

Justification:

- Converts data into long format using `melt()`.
- Uses `dcast()` with `fun.aggregate = mean` to calculate average values.
- Uses `dcast()` with `fun.aggregate = var` to calculate variance.
- Resolves duplicate records during wide-format conversion and preserves data accuracy.

Data Frames

Q1. Analyze a dataset of 50,000 student records stored in a data frame. Identify the most efficient method to detect duplicate entries and justify your approach.

- Use the `duplicated()` function.
- It quickly identifies duplicate rows.
- It is efficient for large datasets.
- Remove duplicates using `unique()` or subset the data.
- This improves data accuracy and avoids repeated records.

Q2. A data frame contains missing values across multiple columns. Evaluate different strategies for handling missing data and recommend the best solution.

Answer:

- Detect missing values using `is.na()`.
- Remove rows with `na.omit()` if few values are missing.
- Replace missing values using mean, median, or mode.
- Choose the method based on the importance of the data.
- Imputation is recommended when preserving data is important.

Q3. Given a sales data frame with inconsistent data types, analyze the impact on statistical calculations and propose corrective actions.

Answer:

- Incorrect data types cause calculation errors.
- Numeric values stored as characters cannot be summed or averaged.
- Check data types using `str()`.

- Convert using `as.numeric()`, `as.character()`, or `as.factor()`.
- Correct data types ensure accurate analysis.

Q4. Compare two large data frames containing customer transactions. Determine how to identify data integrity issues and evaluate their causes.

Answer:

- Compare using `merge()`, `setdiff()`, or `duplicated()`.
- Check for missing, duplicate, or mismatched records.
- Verify customer IDs and transaction details.
- Causes include data entry errors and incomplete records.
- Clean and validate data before analysis.

Q5. A company reports conflicting results from the same data frame. Analyze possible reasons and design a validation framework.

Answer:

- Check for missing or duplicate data.
- Verify data types and calculations.
- Ensure consistent filtering and sorting.
- Validate using `summary()`, `str()`, and `head()`.
- Apply data cleaning and verification before reporting.

Making Data Frames

Q1. Design a data frame structure for a smart-city traffic monitoring system and justify the selection of attributes.

Theory

A traffic monitoring data frame should contain attributes such as **Vehicle_ID, Location, Time, Vehicle_Type, Speed, and Signal_Status**. These attributes help monitor traffic flow, detect congestion, and improve traffic management. A well-structured data frame makes analysis efficient.

```

>
> traffic <- data.frame(
+   Vehicle_ID = c(101,102,103),
+   Location = c("A","B","C"),
+   Time = c("10:00","10:05","10:10"),
+   Vehicle_Type = c("Car","Bus","Bike"),
+   Speed = c(60,40,55),
+   Signal_Status = c("Green","Red","Green")
+ )
>
> print(traffic)
  Vehicle_ID Location Time Vehicle_Type Speed Signal_Status
1        101         A 10:00         Car    60         Green
2        102         B 10:05         Bus    40          Red
3        103         C 10:10         Bike    55         Green
>

```

Q2. Evaluate two alternative data frame designs for storing IoT sensor data and recommend the most scalable model.

Theory

IoT data can be stored in **multiple data frames** or **a single data frame with Sensor_ID**. A single data frame is more scalable because it is easier to manage, filter, and analyze.

Program

```

<
> iot <- data.frame(
+   Sensor_ID = c("s1","s2","s3"),
+   Temperature = c(28,30,27),
+   Humidity = c(65,70,68)
+ )
>
> print(iot)
  Sensor_ID Temperature Humidity
1        s1           28        65
2        s2           30        70
3        s3           27        68
<

```

Q3. Analyze the consequences of assigning incorrect data types while creating a data frame for healthcare records.

Theory

Incorrect data types can produce calculation errors. Numeric values stored as characters cannot be used in statistical analysis. Correct data types improve data accuracy and reliability.

Program

```

<
> health <- data.frame(
+   Patient = c("P1","P2"),
+   Age = c("25","30")
+ )
>
> str(health)
'data.frame':  2 obs. of  2 variables:
 $ Patient: chr  "P1" "P2"
 $ Age    : chr  "25" "30"
>
> health$Age <- as.numeric(health$Age)
>
> str(health)
'data.frame':  2 obs. of  2 variables:
 $ Patient: chr  "P1" "P2"
 $ Age    : num  25 30
>

```

Q4. Create a strategy for constructing a data frame from multiple heterogeneous data sources while preserving consistency.

Theory

Standardize column names and data types, remove duplicates, handle missing values, and merge using a common key. This preserves data consistency and improves analysis.

Program

```

> df1 <- data.frame(ID=c(1,2), Name=c("A","B"))
> df2 <- data.frame(ID=c(1,2), Marks=c(80,90))
>
> result <- merge(df1, df2, by="ID")
>
> print(result)
  ID Name Marks
1  1   A     80
2  2   B     90

```

Q5. Assess the efficiency of creating a data frame from large vectors versus importing directly from files.

Theory

Creating data frames from vectors is suitable for small datasets. Importing data from files (CSV, Excel) is faster and more efficient for large datasets because it reduces manual work and improves memory management.

Program

```

> student <- data.frame(
+   ID = c(1,2,3),
+   Name = c("Ravi","Priya","Arun"),
+   Marks = c(85,90,78)
+ )
>
> print(student)
  ID Name Marks
1  1  Ravi    85
2  2 Priya    90
3  3  Arun    78

```

Working with Data Frames

Q1. Analyze a scenario where filtering operations on a large data frame are producing unexpected results. Determine the root cause.

Theory

Unexpected filtering results may occur due to incorrect conditions, missing values (NA), wrong data types, or incorrect column names. Check the data using `str()`, `summary()`, and `is.na()` before filtering.

```
> df <- data.frame(
+   Name = c("A", "B", "C"),
+   Marks = c(80, NA, 90)
+ )
>
> result <- df[df$Marks > 75, ]
>
> print(result)
  Name Marks
1    A    80
NA <NA>   NA
3    C    90
```

Q2. Evaluate different methods for sorting a million-row data frame and justify the most efficient approach.

Theory

Sorting can be performed using `order()`. It is efficient for large datasets and arranges rows in ascending or descending order.

Program

```
> df <- data.frame(
+   Name = c("A", "B", "C"),
+   Marks = c(80, 95, 70)
+ )
>
> sorted_df <- df[order(df$Marks), ]
>
> print(sorted_df)
  Name Marks
3    C    70
1    A    80
2    B    95
```

Q3. A data analyst performs multiple transformations on a data frame. Analyze how errors can propagate through the workflow.

Theory

Errors in one transformation can affect all subsequent operations. Validate data after each step using `str()`, `summary()`, and `head()` to avoid incorrect results.

Program

```

> df <- data.frame(
+   Marks = c("80", "90", "85")
+ )
>
> df$Marks <- as.numeric(df$Marks)
>
> print(df)
  Marks
1    80
2    90
3    85

```

Q4. Compare indexing and logical subsetting techniques for extracting data and evaluate their performance trade-offs.

Theory

Indexing extracts rows by position, while logical subsetting extracts rows based on conditions. Logical subsetting is more flexible for data analysis.

Program

```

> df <- data.frame(
+   Name = c("A", "B", "C"),
+   Marks = c(80, 90, 70)
+ )
>
> print(df[2, ])
  Name Marks
2    B    90
> print(df[df$Marks > 75, ])
  Name Marks
1    A    80
2    B    90

```

Q5. Design a workflow to monitor and validate modifications made to a shared data frame by multiple users.

Theory

Keep a backup of the original data frame, compare changes after modifications, validate using `summary()` and `str()`, and maintain version history to track updates.

```

<
> df <- data.frame(
+   ID = c(1, 2),
+   Marks = c(80, 90)
+ )
>
> backup <- df
>
> df$Marks[1] <- 85
>
> print(backup)
  ID Marks
1  1    80
2  2    90
> print(df)
  ID Marks
1  1    85
2  2    90

```

Data Reshaping

Q1. Analyze a business dataset before and after reshaping. Evaluate how the transformation affects data interpretation.

Answer:

- Before reshaping, data is in **wide format** with multiple variables in separate columns.
- After reshaping, data is converted to **long format**, where each row represents one observation.
- Long format simplifies filtering, grouping, and visualization.
- It improves statistical analysis and data interpretation.

Q2. A dataset must be reshaped for machine learning. Determine which structure is most suitable and justify your decision.

Answer:

- A **long/tidy format** is most suitable.
- Each row should represent one observation.
- Each column should represent one feature.
- This structure improves model training, preprocessing, and prediction accuracy.

Q3. Evaluate the risks of reshaping data incorrectly in a healthcare analytics application.

Answer:

- Incorrect reshaping may cause data loss.
- Patient records may become mismatched or duplicated.
- Statistical results may become inaccurate.
- Wrong clinical decisions may be made.
- Proper validation should be performed after reshaping.

Q4. Compare wide and long data formats for time-series analysis and recommend the better approach for forecasting.

Wide Format	Long Format
Data stored in multiple columns	One observation per row
Difficult to analyze over time	Easy for time-series analysis
Less flexible	More flexible
Recommended: Long format for forecasting because it supports easier analysis and visualization.	

Q5. Design a reshaping strategy for integrating survey responses collected across multiple years.

Melting and Casting of Data

Answer:

- Standardize column names across all years.
- Convert all datasets into the same format.
- Merge the datasets using a common key (e.g., Year or Respondent ID).
- Remove duplicate records.
- Validate the final dataset before analysis.

Melting and Casting of Data

Q1. Analyze a dataset where repeated measurements are stored in multiple columns. Determine how melting improves analysis.

Answer:

- Repeated measurements in multiple columns are difficult to analyze.
- **Melting** converts wide data into long format.
- Each row represents a single observation.
- It simplifies filtering, grouping, and visualization.
- It improves statistical analysis and reporting.

Q2. Evaluate the impact of casting data into different summary structures for business reporting.

Answer:

- **Casting** converts long data into a summarized wide format.
- It provides reports such as totals, averages, and counts.
- Different casting methods produce different summaries.
- It improves decision-making and business reporting.
- Choose the aggregation function based on the reporting requirement

Q3. Compare two casting approaches that produce different aggregation results and explain the reasons.

Answer:

- One approach uses **sum()** to calculate total values.
- Another uses **mean()** to calculate average values.
- Different aggregation functions produce different results.
- The choice depends on the analysis objective.
- Select the appropriate function for accurate reporting.

Q4. Design a workflow using melting and casting to prepare data for predictive modeling.

Answer:

- Convert wide data to long format using **melt()**.
- Clean the data by removing duplicates and handling missing values.
- Perform feature engineering if required.
- Summarize data using **dcast()**.
- Use the prepared dataset for predictive modeling.

Q5. Assess the challenges of applying melt and cast operations on large-scale industrial datasets.

Answer:

- Large datasets require more memory and processing time.

- Duplicate records can create aggregation issues.
- Missing values may affect results.
- Incorrect reshaping can lead to data loss.
- Efficient data cleaning and validation improve performance.

Merging Data Frames

Q1. Analyze the consequences of performing an incorrect join operation between two financial datasets.

Answer:

- Incorrect joins may create missing or duplicate records.
- Financial calculations may become inaccurate.
- Reports may show incorrect balances or transactions.
- Data integrity and decision-making are affected.
- Always verify the join key before merging.

Q2. Evaluate inner, outer, left, and right joins for integrating customer and transaction data.

Answer:

- **Inner Join:** Returns only matching records from both tables.
- **Left Join:** Returns all records from the left table and matching records from the right table.
- **Right Join:** Returns all records from the right table and matching records from the left table.
- **Outer Join:** Returns all records from both tables, including unmatched records.
- The choice depends on the data integration requirement.

Q3. A merged data frame contains unexpected duplicate rows. Investigate the possible causes and propose solutions.

Answer:

- Duplicate rows may occur due to duplicate key values.
- Incorrect join columns can also create duplicates.
- Check for duplicate records before merging.
- Remove duplicates using `duplicated()` or `unique()`.
- Use appropriate join keys to ensure accurate merging.

Q4. Design a strategy to merge data frames from multiple departments while ensuring data consistency.

Answer:

- Use a common key such as Employee ID or Department ID.
- Standardize column names and data types.
- Remove duplicate records.
- Handle missing values before merging.
- Validate the merged data frame after integration.

Q5. Compare key-based merging and index-based merging techniques for large engineering datasets.

Key-Based Merging	Index-Based Merging
Uses common key columns	Uses row positions (indices)
More accurate and reliable	Faster but less reliable
Suitable for large real-world datasets	Suitable only when row order matches
Recommended: Key-based merging because it preserves data integrity and reduces matching errors.	

RUBRICS FOR CALCULATION

Class Test					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Data Frame Concepts	25	Demonstrates thorough understanding of Data Frame concepts and operations.	Demonstrates good understanding with minor omissions.	Demonstrates basic understanding with limited details.	Demonstrates poor or incorrect understanding of Data Frame concepts.
Understanding of Reshaping and Merging Techniques	25	Clearly explains reshaping and merging concepts with appropriate use cases.	Explains concepts correctly with minor gaps.	Provides partial understanding of reshaping or merging concepts.	Unable to explain reshaping or merging concepts correctly.
Knowledge of File I/O Operations	20	Accurately explains file handling functions and their usage.	Explains most file handling concepts correctly.	Provides limited explanation of file operations.	Demonstrates little understanding of file I/O operations.
Application to Data Analysis and Dataset Preparation	20	Effectively applies concepts to data preparation and analysis tasks.	Provides appropriate applications with adequate explanation.	Gives limited applications or examples.	Unable to relate concepts to practical applications.
Technical Accuracy and Completeness	10	Uses correct terminology and provides complete answers.	Mostly accurate with minor technical errors.	Some inaccuracies and incomplete responses.	Major technical errors and incomplete answers.